

1. The Interactive Math & Control Flow Engine

Write a C++ program that utilizes a do-while loop to keep a console application running until the user chooses to exit. Inside the loop, prompt the user to input two numbers and choose an operation via a switch statement:

- Calculate the hypotenuse of a right triangle (using the two numbers as sides and the `<cmath>` library).
- Perform a temperature conversion (treating the first number as Celsius to convert to Fahrenheit).
- Generate a random number between the two input values.

After displaying the result of the chosen operation, use a ternary operator to print whether the final calculated value is positive or negative.

2. Dynamic Memory & Array Manipulation

Write a program that asks the user how many items they want to store, then use dynamic memory allocation to create an integer array of that size.

- Populate this array using a random number generator.
- Create a user-defined function that accepts this array (using pointers) and its size, and sorts it in ascending order.
- Create a second function that asks the user for a number, searches the sorted array for that specific element, and returns its memory address (or a null pointer if it doesn't exist).
- Ensure you free the dynamically allocated memory at the end of the program.

3. Templates, Overloading & Memory Optimization

Demonstrate advanced function mechanics by writing a program with the following constraints:

- Create a function template that can swap the values of two variables of any data type (int, double, char).
- Write two overloaded functions named `calculateDiscount`.
- The first version should take a standard double by value representing a price.

- The second version should take a const reference to a `std::string` (representing a promo code) and a double passed by reference. Modify the referenced double directly inside the function based on the promo code.
- Demonstrate variable scope by using a global variable for the baseline tax rate and a local variable for the final price inside your main function.

4. Structs, Enums & Recursion

Topics covered: Structs, Pass structs as arguments, Enums, Recursion, Strings.

Define an enum representing the four seasons (SPRING, SUMMER, FALL, WINTER). Then, define a struct called Crop containing a `std::string` for the crop's name, an int for its growth time, and the season enum for when it should be planted.

- Create an array of three Crop structs and pass this array to a function that prints out their details.
- Write a separate recursive function called `calculateYield` that takes an integer days and a `dailyGrowthRate`. The function should recursively calculate and return the total plant height after the given number of days.

5. Encapsulation & Secure State Management

Design a `UserAccount` class to represent a secure profile.

- Include private attributes for username, password, and `accountBalance`.
- Implement constructor overloading: provide a default constructor that sets default values, and a parameterized constructor that accepts a username and a starting balance.
- Implement strict getters and setters. The setter for the password must require the new password to be at least 8 characters long. The setter for `accountBalance` must prevent the balance from being set to a negative number directly.

6. Hierarchical Architecture

Implement a class hierarchy demonstrating inheritance.

- Create a base class called `Appliance` with protected attributes `brand` and `powerConsumption` (in watts), and a public method `turnOn()` that prints a generic startup message.

- Create two derived classes: WashingMachine and Microwave.
- WashingMachine should inherit from Appliance and add a specific attribute drumCapacity. Add a method startWashCycle() unique to this class.
- Microwave should inherit from Appliance and add an attribute maxHeatingTime. Add a method defrost() unique to this class.
- In your main function, instantiate both derived classes, initialize their inherited and unique properties, and call their respective methods to prove the inheritance chain works.

BONUS TASK

Problem Statement: Implement a Queue using a Singly Linked List

Design and implement a Queue data structure using a singly linked list. You must maintain two pointers, front and rear, to ensure that both insertion (enqueue) and deletion (dequeue) operations run in **O(1)** time complexity.

Requirements

Implement the following classes and methods:

1. A Node struct/class:

- data: An integer value.
- next: A pointer to the next node.

2. A Queue class:

- front: A pointer pointing to the first node in the queue.
- rear: A pointer pointing to the last node in the queue.

3. Required Methods:

- void enqueue(int x): Creates a new node with value x and inserts it at the rear of the queue.
- int dequeue(): Removes the node from the front of the queue and returns its value. If the queue is empty, return -1.
- int getFront(): Returns the value of the front node without removing it. If the queue is empty, return -1.
- bool isEmpty(): Returns true if the queue is empty, and false otherwise.

